

Q164

LC 643

Google

Amazon

Max Average Subarray - I

Key Observations:

- Window size is fixed to be $= k$.
- We need the sum of every window.
- **Maximizing average = Maximizing sum.**

1. Optimal Approach (Sliding Window)

Time Complexity: $O(n)$ | Space Complexity: $O(1)$

```
for (int i = 0; i < k; i++) {
    window_sum += nums[i];
}
maxsum = window_sum;

for (int j = k; j < n; j++) {
    window_sum += nums[j];
    window_sum -= nums[j - k];
    maxsum = max(maxsum, window_sum);
}
return (double)maxsum / k;
```

2. Prefix Sum Approach

Time Complexity: $O(n)$ | Space Complexity: $O(n)$

```
// Step 1: Build Prefix Sum Array
vector<long long> prefix(n + 1, 0);
for (int i = 0; i < n; i++) {
    prefix[i + 1] = prefix[i] + nums[i];
}
long long maxsum = LLONG_MIN;

// Step 2: Check for every window of length k
for (int l = 0; l + k - 1 < n; l++) {
    int r = l + k - 1;
    long long windowsum = prefix[r + 1] - prefix[l];

    maxsum = max(maxsum, windowsum);
}
return (double)maxsum / k;
```

Why `prefix[r + 1] - prefix[l]`?

If you need the sum from index 1 to 3, you compute `prefix[4] - prefix[1]`.

0	1	2	3	4	
---	---	---	---	---	--



3. Brute Force Approach

Time Complexity: $O(n * k)$ | Space Complexity: $O(1)$

For every starting position, iterate for the next k elements to calculate the sum.